AST-basierte Interpreter: Basics

Carsten Gips (HSBI)

TL;DR
Ein AST-basierter Interpreter besteht oft aus einem "Visitor-Dispatcher": Man traversiert mit einer `eval()`-Funktion den AST und ruft je nach Knotentyp die passende Funktion auf. Dabei werden bei Ausdrücken (*Expressions*) Werte berechnet und zurückgegeben, d.h. hier hat man einen Rückgabewert und ein entsprechendes **`return`** im **`switch`**/**`case`**, während man bei Anweisungen (*Statements*) keinen Rückgabewert hat.
Der Wert von Literalen ergibt sich direkt durch die Übersetzung des jeweiligen Werts in den passenden Typ der Implementierungssprache. Bei Ausdrücken interpretiert zunächst die Teilausdrücke durch den Aufruf von `eval()` für die jeweiligen AST-Kindknoten und berechnet daraus das gewünschte Ergebnis.
Für Blöcke und Variablen muss man analog zum Aufbau von Symboltabellen wieder Scopes berücksichtigen, d.h. man benötigt Strukturen ähnlich zu den Symboltabellen (hier "Umgebung" (*Environment*) genannt). Es gibt eine globale Umgebung, und mit dem Betreten eines neuen Blocks wird eine neue Umgebung aufgemacht, deren Eltern-Umgebung die bisherige Umgebung ist.
Zu jedem Namen kann man in einer Umgebung einen Wert definieren bzw. abrufen. Dabei muss man je nach Semantik der zu interpretierenden Sprache unterscheiden zwischen der "Definition" und der "Zuweisung" einer Variablen: Die Definition erfolgt i.d.R. in der aktuellen Umgebung, bei der Zuweisung sucht man ausgehend von der aktuellen Umgebung bis hoch zur globalen Umgebung nach dem ersten Vorkommen der Variablen und setzt den Wert in der gefundenen Umgebung. Bei Sprachen, die Variablen beim ersten Zugriff definieren, muss man dieses Verhalten entsprechend anpassen.
Videos

- VL AST-basierte Interpreter (Basics)

---

1 Aufgaben im Interpreter

Im Allgemeinen reichen einfache syntaxgesteuerte Interpreter nicht aus. Normalerweise simuliert ein Interpreter die Ausführung eines Programms durch den Computer. D.h. der Interpreter muss über die entsprechenden Eigenschaften verfügen: Prozessor, Code-Speicher, Datenspeicher, Stack …

```
int x = 42;
int f(int x) {
    int y = 9;
    return y+x;
}

x = f(x);
```

- Aufbauen des AST … => Lexer+Parser
- Auflösen von Symbolen/Namen … => Symboltabellen, Resolving
- Type-Checking und -Inference … => Semantische Analyse (auf Symboltabellen)

- Speichern von Daten: Name+Wert vs. Adresse+Wert (Erinnerung: Data-Segment und Stack im virtuellen Speicher)

- Ausführen von Anweisungen Text-Segment im virtuellen Speicher; hier über den AST
- Aufruf von Funktionen und Methoden Kontextwechsel nötig: Was ist von wo aus sichtbar?

2 AST-basierte Interpreter: Visitor-Dispatcher

```
def eval(self, AST t):
    if   t.type == Parser.BLOCK  : block(t)
    elif t.type == Parser.ASSIGN : assign(t)
    elif t.type == Parser.RETURN : ret(t)
    elif t.type == Parser.IF     : ifstat(t)
    elif t.type == Parser.CALL   : return call(t)
    elif t.type == Parser.ADD    : return add(t)
    elif t.type == Parser.MUL    : return mul(t)
    elif t.type == Parser.INT    : return Integer.parseInt(t.getText())
    elif t.type == Parser.ID     : return load(t)
    else : ...  # catch unhandled node types
    return None;
```

Nach dem Aufbau des AST durch Scanner und Parser und der semantischen Analyse anhand der Symboltabellen müssen die Ausdrücke (*expressions*) und Anweisungen (*statements*) durch den Interpreter ausgewertet werden. Eine Möglichkeit dazu ist das Traversieren des AST mit dem Visitor-Pattern. Basierend auf dem Typ des aktuell betrachteten AST-Knotens wird entschieden, wie damit umgegangen werden soll. Dies erinnert an den Aufbau der Symboltabellen …

Die `eval()`-Methode bildet das Kernstück des (AST-traversierenden) Interpreters. Hier wird passend zum aktuellen AST-Knoten die passende Methode des Interpreters aufgerufen.

Hinweis: Im obigen Beispiel wird nicht zwischen der Auswertung von Ausdrücken und Anweisungen unterschieden, es wird die selbe Methode `eval()` genutzt. Allerdings liefern Ausdrücke einen Wert zurück (erkennbar am **`return`** im jeweiligen **`switch`**/**`case`**-Zweig), während Anweisungen keinen Wert liefern.

In den folgenden Beispielen wird davon ausgegangen, dass ein komplettes Programm eingelesen, geparst, vorverarbeitet und dann interpretiert wird.

Für einen interaktiven Interpreter würde man in einer Schleife die Eingaben lesen, parsen und vorverarbeiten und dann interpretieren. Dabei würde jeweils der AST und die Symboltabelle *ergänzt*, damit die neuen Eingaben auf frühere verarbeitete Eingaben zurückgreifen können. Durch die Form der Schleife "Einlesen – Verarbeiten – Auswerten" hat sich auch der Name "*Read-Eval-Loop*" bzw. "*Read-Eval-Print-Loop*" (**REPL**) eingebürgert.

3 Auswertung von Literalen und Ausdrücken

- Typen mappen: Zielsprache => Implementierungssprache

  Die in der Zielsprache verwendeten (primitiven) Typen müssen auf passende Typen der Sprache, in der der Interpreter selbst implementiert ist, abgebildet werden.

  Beispielsweise könnte man den Typ `nil` der Zielsprache auf den Typ **`null`** des in Java implementierten Interpreters abbilden, oder den Typ `number` der Zielsprache auf den Typ `Double` in Java mappen.

- Literale auswerten:

```
INT: [0-9]+ ;
```

```
elif t.type == Parser.INT : return Integer.parseInt(t.getText())
```

Das ist der einfachste Teil … Die primitiven Typen der Zielsprache, für die es meist ein eigenes Token gibt, müssen als Datentyp der Interpreter-Programmiersprache ausgewertet werden.

- Ausdrücke auswerten:

```
add: e1=expr "+" e2=expr ;
```

```
def add(self, AST t):
    lhs = eval(t.e1())
    rhs = eval(t.e2())
    return (double)lhs + (double)rhs  # Semantik!
```

Die meisten möglichen Fehlerzustände sind bereits durch den Parser und bei der semantischen Analyse abgefangen worden. Falls zur Laufzeit die Auswertung der beiden Summanden keine Zahl ergibt, würde eine Java-Exception geworfen, die man an geeigneter Stelle fangen und behandeln muss. Der Interpreter soll sich ja nicht mit einem Stack-Trace verabschieden, sondern soll eine Fehlermeldung präsentieren und danach normal weiter machen …

4 Kontrollstrukturen

```
ifstat: 'if' expr 'then' s1=stat ('else' s2=stat)? ;
```

```
def ifstat(self, AST t):
    if eval(t.expr()): eval(t.s1())
    else:
        if t.s2(): eval(t.s2())
```

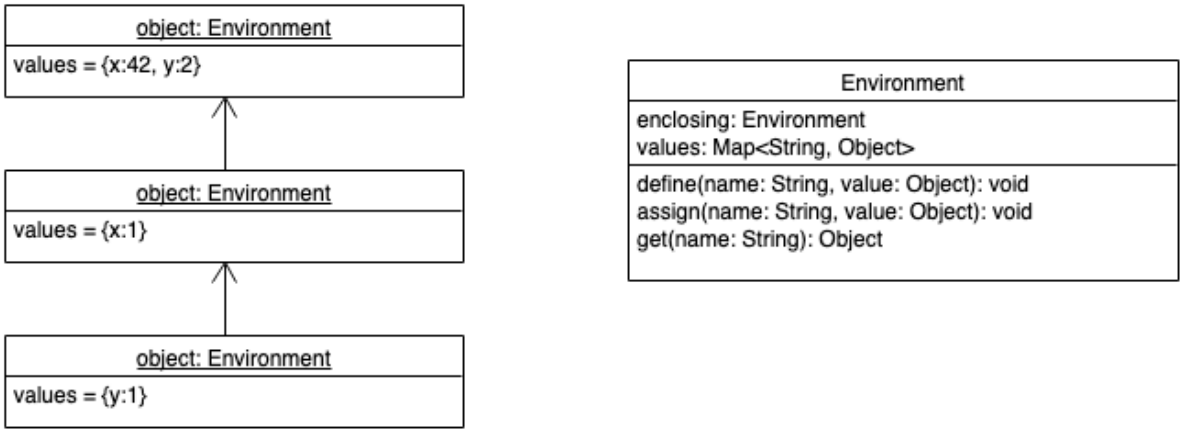
Analog können die anderen bekannten Kontrollstrukturen umgesetzt werden, etwa `switch`/`case`, `while` oder `for`.

Dabei können erste Optimierungen vorgenommen werden: Beispielsweise könnten `for`-Schleifen im Interpreter in `while`-Schleifen transformiert werden, wodurch im Interpreter nur ein Schleifenkonstrukt implementiert werden müsste.

5 Zustände: Auswerten von Anweisungen

```
int x = 42;
float y;
{
    int x;
```

```
5       x = 1;
6       y = 2;
7       { int y = x; }
8   }
```



Das erinnert nicht nur zufällig an den Aufbau der Symboltabellen :-)

Und so lange es nur um Variablen ginge, könnte man die Symboltabellen für das Speichern der Werte nutzen. Allerdings müssen wir noch Funktionen und Strukturen bzw. Klassen realisieren, und spätestens dann kann man die Symboltabelle nicht mehr zum Speichern von Werten einsetzen. Also lohnt es sich, direkt neue Strukturen für das Halten von Variablen und Werten aufzubauen.

6 Detail: Felder im Interpreter

Eine mögliche Implementierung für einen Interpreter basierend auf einem ANTLR-Visitor ist nachfolgend gezeigt.

Hinweis: Bei der Ableitung des `BaseVisitor<T>` muss der Typ `T` festgelegt werden. Dieser fungiert als Rückgabetyp für die Visitor-Methoden. Entsprechend können alle Methoden nur einen gemeinsamen (Ober-) Typ zurückliefern, weshalb man sich an der Stelle oft mit `Object` behilft und dann manuell den konkreten Typ abfragen und korrekt casten muss.

```
1  class Interpreter(BaseVisitor<Object>):
2      __init__(self, AST t):
3          BaseVisitor<Object>.__init__(self)
4          self.root = t
5          self.env = Environment()
```

Quelle: Eigener Code basierend auf einer Idee nach Interpreter.java by Bob Nystrom on Github.com (MIT)

7 Ausführen einer Variablendeklaration

```
varDecl: "var" ID ("=" expr)? ";" ;
```

```
def varDecl(self, AST t):
    # deklarierte Variable (String)
    name = t.ID().getText()

    value = None;  # TODO: Typ der Variablen beachten (Defaultwert)
    if t.expr(): value = eval(t.expr())

    self.env.define(name, value)

    return None
```

Wenn wir bei der Traversierung des AST mit `eval()` bei einer Variablendeklaration vorbeikommen, also etwa `int x;` oder `int x = wuppie + fluppie;`, dann wird im **aktuellen** Environment der String "x" sowie der Wert (im zweiten Fall) eingetragen.

8 Ausführen einer Zuweisung

```
assign: ID "=" expr;
```

```
def assign(self, AST t):
    lhs = t.ID().getText()
    value = eval(t.expr())

    self.env.assign(lhs, value)  # Semantik!
}

class Environment:
    def assign(self, String n, Object v):
        if self.values[n]: self.values[n] = v
        elif self.enclosing: self.enclosing.assign(n, v)
        else: raise RuntimeError(n, "undefined variable")
```

Quelle: Eigener Code basierend auf einer Idee nach Environment.java by Bob Nystrom on Github.com (MIT)

Wenn wir bei der Traversierung des AST mit `eval()` bei einer Zuweisung vorbeikommen, also etwa `x = 7;` oder `x = wuppie + fluppie;`, dann wird zunächst im aktuellen Environment die rechte Seite der Zuweisung ausgewertet (Aufruf von `eval()`). Anschließend wird der Wert für die Variable im Environment eingetragen: Entweder sie wurde im aktuellen Environment früher bereits definiert, dann wird der neue Wert hier eingetragen. Ansonsten wird entlang der Verschachtelungshierarchie gesucht und entsprechend eingetragen. Falls die Variable nicht gefunden werden kann, wird eine Exception ausgelöst.

An dieser Stelle kann man über die Methode `assign` in der Klasse `Environment` dafür sorgen, dass nur bereits deklarierte Variablen zugewiesen werden dürfen. Wenn man stattdessen wie etwa in Python das implizite Erzeugen neuer Variablen erlaubten möchte, würde man statt `Environment#assign` einfach `Environment#define` nutzen …

Anmerkung: Der gezeigte Code funktioniert nur für normale Variablen, nicht für Zugriffe auf Attribute einer Struct oder Klasse!

9 Blöcke: Umgang mit verschachtelten Environments

```
block:  '{' stat* '}' ;
```

```
def block(self, AST t):
    prev = self.env

    try:
        self.env = Environment(self.env)
        for s in t.stat(): eval(s)
    finally: self.env = prev

    return None;
```

Quelle: Eigener Code basierend auf einer Idee nach Interpreter.java by Bob Nystrom on Github.com (MIT)

Beim Interpretieren von Blöcken muss man einfach nur eine weitere Verschachtelungsebene für die Environments anlegen und darin dann die Anweisungen eines Blockes auswerten …

Wichtig: Egal, was beim Auswerten der Anweisungen in einem Block passiert: Es muss am Ende die ursprüngliche Umgebung wieder hergestellt werden (`finally`-Block).

10 Wrap-Up

- Interpreter simulieren die Programmausführung
  - Namen und Symbole auflösen
  - Speicherbereiche simulieren
  - Code ausführen: Read-Eval-Loop
- Traversierung des AST: `eval(AST t)` als Visitor-Dispatcher
- Scopes mit `Environment` (analog zu Symboltabellen)
- Interpretation von Blöcken und Variablen (Deklaration, Zuweisung)

11 Zum Nachlesen

- Nystrom (2021): Kapitel: A Tree-Walk Interpreter, insb. 8. Statements and State
- Grune u. a. (2012): Kapitel 6
- Mogensen (2017): Kapitel 4

Lernziele

- k3: Traversierung von Parse-Trees und Implementierung von Aktionen mit Hilfe des Visitor-Patterns
- k3: Interpreter müssen Namen und Werte speichern: Environment-Strukturen analog zu den Symboltabellen
- k3: Code-Ausführung im Interpreter durch eine Read-Eval-Schleife: Implementierung mit einem Visitor

---

Quellen

Grune, D., K. van Reeuwijk, H. E. Bal, C. J. H. Jacobs, und K. Langendoen. 2012. *Modern Compiler Design*. Springer.
Mogensen, T. 2017. *Introduction to Compiler Design*. Springer. https://doi.org/10.1007/978-3-319-66966-3.
Nystrom, R. 2021. *Crafting Interpreters*. Genever Benning. https://github.com/munificent/craftinginterpreters.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- Eigener Code basierend auf einer Idee nach Environment.java by Bob Nystrom on Github.com (MIT)
- Eigener Code basierend auf einer Idee nach Interpreter.java by Bob Nystrom on Github.com (MIT)

> **Last modified:** a10718a (Remove Hugo: tooling: shift headings (use h1 as top-level headings), 2025-07-25)